

Server Side Scripting with Database Connectivity

PHP(8 Hrs.): Introduction to Server Side Scripting Language, PHP introduction. Basic PHP Syntax, Comments in PHP, Variables, PHP Operators, Control Structures(if else, switch, all loops), Arrays, For Each Loop, Functions, Form Handling, PHP \$_GET, PHP \$_POST, PHP \$_REQUEST, PHP date () Function, PHP include File, File Handling, File Uploading, PHP Sessions, Sending Emails, PHP Cookies.

MySQL(7 Hrs.): Introduction to MySQL, PHP MySQL Connect to a Database, Closing a Connection, MySQL Data Types, MySQL Insert, MySQL Select, MySQL Where Clause, MySQL Delete, MySQL Update, MySQL Aggregate Functions(sum, avg, count etc); MySQL Order by and Group by Clause, MySQL Subqueries, MySQL Joins

Dynamic content generation

- There are two ways to provide dynamic content:
 1. Using client-side scripting and frameworks such as JavaScript, AJAX and Bootstrap to change the UI behavior within a specific Web page in response to specific user actions and timings.
 - This gives dynamic behavior to the UI presentation. This is normally used in Web applications and interactive websites.
 2. Using server-side scripting and processing to dynamically form both the layout and content to be delivered to the user based on parameters such as the user's location, time of day, browser being used or user preferences.
 - Some good examples of this are social networking sites and content delivery sites.
 - Social networking sites such as Facebook and Twitter provide entirely different content per user because of the difference of their connections and subscribed services, while sites like YouTube and Amazon provide dynamic content based on user-specific preferences based on past purchases or views; the server gives suggested items or content that the user may like based on historical data.

Server-side Scripting languages

- We can create web content by using Server-side Scripting languages like PHP, ASP.Net, C#, Python, Java etc. Here we are going to discuss PHP Server-side scripting.
- What can PHP do? (Application)
 - Generate pages and files dynamically.
 - Create, open, read, write and close files on the server.
 - Collect data from a web form such as user information, email, phone number, etc.
 - Send emails to the users of your website.
 - Send and receive cookies to track the visitor of your website.
 - Store, delete, and modify information in your database.
 - Restrict unauthorized access to your website.
 - Encrypt data for safe transmission over internet.

Introduction to PHP

- PHP is a recursive acronym for "PHP: Hypertext Preprocessor".
- PHP is a server-side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.
- It is integrated with a number of popular databases, including MySQL, PostgreSQL, Oracle, Sybase, Informix, and Microsoft SQL Server.
- PHP is pleasingly zippy in its execution, especially when compiled as an Apache module on the Unix side. The MySQL server, once started, executes even very complex queries with huge result sets in record-setting time.
- PHP supports a large number of major protocols such as POP3, IMAP, and LDAP. PHP4 added support for Java and distributed object architectures (COM and CORBA), making n-tier development a possibility for the first time.
- PHP Syntax is C-Like.

Advantages of PHP over Other Languages

- Easy to learn – PHP is easy to learn and use. For beginner programmers who just started out in web development, PHP is often considered as the preferable choice of language to learn.
- Open source – PHP is an open-source project. It is developed and maintained by a worldwide community of developers who make its source code freely available to download and use.
- Portability – PHP runs on various platforms such as Microsoft Windows, Linux Mac OS, etc. and it is compatible with almost all servers used today such as Apache, IIS, etc.
- Fast Performance – Scripts written in PHP usually execute or runs faster than those written in other scripting languages like ASP, Ruby, Python, Java, etc.
- Vast Community – Since PHP is supported by the worldwide community, finding help or documentation related to PHP online is extremely easy.

PHP Syntax

- PHP has quite easy syntax, if you are familiar with any C-type language.
- It has all the same structures that you are familiar with other programming languages.
- A PHP script can be placed anywhere in the document.
- It starts with `<?php` and ends with `?>`

```
<?php  
//PHP codes;  
?>
```

- The default file extension for PHP files is “.php”.
- Example:

```
<html>  
<body>  
<h1>My first PHP page</h1>  
<?php  
echo "HelloWorld!";  
echo "<br>";  
print "<br>";  
print "This is my first program in PHP.";  
?>  
</body>  
</html>
```

Comments in PHP

- To write a single-line comment either start the line with two slashes (//) or a hash symbol (#).
- However, to write multi-line comments, start the comment with a slash followed by an asterisk (/*) and end the comment with an asterisk followed by a slash (*/).

```
<?php
```

```
//This is a single line comment
```

```
#This is also a single line comment
```

```
/*
```

```
This is a multiple line comment block  
that spans across more than  
one line
```

```
*/
```

```
echo "Hello, world!";
```

```
?>
```

Case Sensitivity in PHP

- Variable names in PHP are case-sensitive.
- As a result, the variables `$color`, `$Color`, and `$COLOR` are treated as three different variables.
- However, the keywords, functions and classes name are case-insensitive.
- As a result, calling the `gettype()` or `GETTYPE()` produce the same result.

PHP Variables

- In PHP, a variable does not need to be declared before adding a value to it.
- PHP automatically converts the variable to the correct data type, depending on its value.
- After declaring a variable, it can be reused throughout the code.
- The assignment operator (=) is used to assign value to a variable.
- In PHP, variable can be declared as

```
$var_name = value;
```

- Example:

```
<?php  
$txt = "This product cost 50. "  
$x = 5;  
$y = 10.5;  
echo $txt;  
echo "<br>";  
echo $x;  
echo "<br>";  
echo $y;  
?>
```

Naming Conventions for PHP Variables

- All variables in PHP start with a \$ sign, followed by the name of the variable.
- A variable name must start with a letter or the underscore character _.
- A variable name cannot start with a number.
- A variable name in PHP can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- A variable name cannot contain spaces.

Defining a Constant in PHP

- A constant is a name or an identifier for a fixed value.
- Constants are like variables, except that once they are defined, they cannot be undefined or changed.
- Constants are very useful for storing data that doesn't change while the script is running.
- Common examples of such data include configuration settings such as database username and password, website's base URL, company name, etc.
- Constants are defined using PHP's `define()` function, which accepts two arguments: the name of the constant, and its value.
- Naming conventions for PHP Constants are similar to that of PHP Variables.

Defining a Constant in PHP (cont...)

- Example:

```
<html>
<body>
<h1>Defining a Constant</h1>
<?php
define("SITE_URL", "https://www.abc.edu.np/");
echo "Thank you for visiting - " . SITE_URL;
?>
</body>
</html>
```

PHP echo & print Statements

- The echo statement can output one or more strings.
- In general terms, the echo statement can display anything that can be displayed to the browser, such as string, numbers, variables values, the results of expressions, etc.
- Like echo, you can also use the print statement to display output to the browser.
- Both echo & print statement works exactly the same way except that the print statement can only output one string, and always returns 1.
- That's why the echo statement is considered marginally faster than the print statement since it doesn't return any value.
- Since echo and print both are language construct and not actually a function (like if statement), you can use it without parentheses. For example. echo or echo() and print or print().

PHP Data Types

- The values assigned to a PHP variable may be of different data types including simple string and numeric types to more complex data types like arrays and objects.
- PHP supports total eight primitive data types:
 - Integer
 - Float
 - String
 - Booleans
 - Array
 - Object
 - Resource &
 - NULL

PHP Strings

- A string is a sequence of letters, numbers, special characters and arithmetic values or combination of all.
- The simplest way to create a string is to enclose the string characters in single or double quotation marks (' or ").
- Strings enclosed in single-quotes are treated almost literally, whereas the strings delimited by the double quotes replaces variables with the string representations of their values as well as specially interpreting certain escape sequences.
- The escape-sequence replacements are:
 - \n is replaced by the newline character.
 - \r is replaced by the carriage-return character.
 - \t is replaced by the tab character.
 - \\$ is replaced by the dollar sign itself (\$).
 - \" is replaced by a single double-quote (").
 - \\ is replaced by a single backslash (\).

PHP Strings – Single & Double Quoted

```
<?php
$my_str = 'World';
// Displays: HelloWorld!
echo "Hello, $my_str!<br>";
// Displays: Hello, $my_str!
echo 'Hello, $my_str!<br>';
// Displays: Hello\tWorld!
echo '<pre>Hello\tWorld!</pre>';
// Displays: Hello World!
echo "<pre>Hello\tWorld!</pre>";
// Displays: I'll be back
echo "I'll be back";
?>
```


Some common string functions

- **Calculating the Length of a String `strlen()`**

```
<?php
$my_str = 'Welcome to PHP classes';
echo "The length of the string is " . strlen($my_str);
?>
```

- **Calculating the Length of a String without space**

```
<?php
$my_str = 'Welcome to PHP classes';
$string_length = strlen($my_str) - substr_count($my_str, ' ');
echo "The length of the string is $string_length";
?>
```

- **Counting Number of Words in a String `str_word_count()`**

```
<?php
$my_str = 'The quick brown fox jumps over the lazy dog.';
echo str_word_count($my_str);
?>
```

Some common string functions

- **Replacing Text within Strings `str_replace()`**

```
<?php
$my_str = 'If the facts do not fit the theory,change the facts.';
echo str_replace("facts", "truth", $my_str, $count);
echo "<br>";
echo "The text was replaced $count times.";
?>
```

- **Reversing a String – `strrev()`**

```
<?php
$my_str = 'You can do anything, but not everything.';
echo strrev($my_str);
?>
```

PHP Operators

- PHP Operator is a symbol i.e used to perform operations on operands. In simple words, operators are used to perform operations on variables or values.
- For example:
 - `$num=10+20;` // + is the operator and 10,20 are operands
- **Arithmetic Operators**
 - The PHP arithmetic operators are used to perform common arithmetic operations such as addition, subtraction, etc. with numeric values.

Arithmetic Operators (Cont...)

Operator	Name	Example	Explanation
+	Addition	<code>\$a + \$b</code>	Sum of operands
-	Subtraction	<code>\$a - \$b</code>	Difference of operands
*	Multiplication	<code>\$a * \$b</code>	Product of operands
/	Division	<code>\$a / \$b</code>	Quotient of operands
%	Modulus	<code>\$a % \$b</code>	Remainder of operands
**	Exponentiation	<code>\$a ** \$b</code>	<code>\$a</code> raised to the power <code>\$b</code>

```
<?php
$x = 10;
$y = 4;
echo($x + $y); // Outputs: 14
echo "<br>";
echo($x - $y); // Outputs: 6
echo "<br>";
echo($x * $y); // Outputs: 40
echo "<br>";
echo($x / $y); // Outputs: 2.5
echo "<br>";
echo($x % $y); // Outputs: 2
echo "<br>";
echo($x ** $y);
?>
```

Assignment Operators

The assignment operators are used to assign value to different variables. The basic assignment operator is "=".

Operator	Name	Example	Explanation
=	Assign	\$a = \$b	The value of right operand is assigned to the left operand.
+=	Add then Assign	\$a += \$b	Addition same as \$a = \$a + \$b
-=	Subtract then Assign	\$a -= \$b	Subtraction same as \$a = \$a - \$b
*=	Multiply then Assign	\$a *= \$b	Multiplication same as \$a = \$a * \$b
/=	Divide then Assign (quotient)	\$a /= \$b	Find quotient same as \$a = \$a / \$b
%=	Divide then Assign (remainder)	\$a %= \$b	Find remainder same as \$a = \$a % \$b

```
<?php
$x = 10;
echo $x;
// Outputs: 10
$x = 20;
$x += 30;
echo $x;
// Outputs: 50
$x = 50;
$x -= 20;
echo $x;
// Outputs: 30
$x = 5;
$x *= 25;
echo $x;
// Outputs: 125
```

```
$x = 50;
$x /= 10;
echo $x;
// Outputs: 5
$x = 100;
$x %= 15;
echo $x;
// Outputs: 10
?>
```

Comparison Operators

Comparison operators allow comparing two values, such as number or string. Below the list of comparison operators are given:

Operator	Name	Example	Explanation
==	Equal	\$a == \$b	Return TRUE if \$a is equal to \$b
===	Identical	\$a === \$b	Return TRUE if \$a is equal to \$b, and they are of same data type
!==	Not identical	\$a !== \$b	Return TRUE if \$a is not equal to \$b, and they are not of same data type
!=	Not equal	\$a != \$b	Return TRUE if \$a is not equal to \$b
<>	Not equal	\$a <> \$b	Return TRUE if \$a is not equal to \$b
<	Less than	\$a < \$b	Return TRUE if \$a is less than \$b
>	Greater than	\$a > \$b	Return TRUE if \$a is greater than \$b
<=	Less than or equal to	\$a <= \$b	Return TRUE if \$a is less than or equal \$b
>=	Greater than or equal to	\$a >= \$b	Return TRUE if \$a is greater than or equal \$b
<=>	Spaceship	\$a <=> \$b	Return -1 if \$a is less than \$b Return 0 if \$a is equal \$b Return 1 if \$a is greater than \$b

Comparison Operators (Cont...)

```
<?php
$x = 25;
$y = 35;
$z = "25";
/*The var_dump() function dumps information about one or more variables.
The information holds type and value of the variable(s).*/
var_dump($x == $z); // Outputs: boolean true
var_dump($x === $z); // Outputs: boolean false
var_dump($x != $y); // Outputs: boolean true
var_dump($x !== $z); // Outputs: boolean true
var_dump($x < $y); // Outputs: boolean true
var_dump($x > $y); // Outputs: boolean false
var_dump($x <= $y); // Outputs: boolean true
var_dump($x >= $y); // Outputs: boolean false
?>
```

Incrementing/Decrementing Operator

The increment and decrement operators are used to increase and decrease the value of a variable.

Operator	Name	Example	Explanation
++	Increment	++\$a	Increment the value of \$a by one, then return \$a
		\$a++	Return \$a, then increment the value of \$a by one
--	decrement	--\$a	Decrement the value of \$a by one, then return \$a
		\$a--	Return \$a, then decrement the value of \$a by one

```
<?php
$x = 10;
echo ++$x; // Outputs: 11
echo $x; // Outputs: 11
$x = 10;
echo $x++; // Outputs: 10
echo $x; // Outputs: 11
$x = 10;
echo --$x; // Outputs: 9
```

```
echo $x; // Outputs: 9
$x = 10;
echo $x--; // Outputs: 10
echo $x; // Outputs: 9
?>
```


Logical Operator

The logical operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
and	And	\$a and \$b	Return TRUE if both \$a and \$b are true
Or	Or	\$a or \$b	Return TRUE if either \$a or \$b is true
xor	Xor	\$a xor \$b	Return TRUE if either \$ or \$b is true but not both
!	Not	! \$a	Return TRUE if \$a is not true
&&	And	\$a && \$b	Return TRUE if either \$a and \$b are true
	Or	\$a \$b	Return TRUE if either \$a or \$b is true

The && and and operators return the same result. The only difference between the && and and operators are their precedencies.

The and operator has higher precedence than the && operator. The precedence of an operator specifies the order in which PHP evaluates.

String Operator

The string operators are used to perform the operation on strings. There are two string operators in PHP, which are given below:

Operator	Name	Example	Explanation
.	Concatenation	<code>\$a . \$b</code>	Concatenate both <code>\$a</code> and <code>\$b</code>
<code>.=</code>	Concatenation and Assignment	<code>\$a .= \$b</code>	First concatenate <code>\$a</code> and <code>\$b</code> , then assign the concatenated string to <code>\$a</code> , e.g. <code>\$a = \$a . \$b</code>

```
<?php
$x = "Hello";
$y = " World!";
echo $x . $y;
// Output: HelloWorld!
$x .= $y;
echo $x;
//Output: HelloWorld!
?>
```

PHP Conditional Statements

- The if statement

```
if (condition){  
    //code to be executed  
}
```

- Example:

```
<?php  
$d = date("D");  
echo $d;  
echo "<br>";  
if($d == "Fri"){  
    echo "Have a nice weekend!";  
}  
?>
```

PHP Conditional Statements (Cont...)

- The if . . . else statement

```
if (condition){  
    //code to be executed if condition is true  
} else{  
    //code to be executed if condition is false  
}
```

- Example:

```
<?php  
$d = date("D");  
if($d == "Fri"){  
    echo "Have a nice weekend!";  
} else{  
    echo "Have a nice day!";  
}  
?>
```

PHP Conditional Statements (Cont...)

- The if . . . elseif . . . else statement

```
if (condition1){  
    //code to be executed if condition1 is true  
} elseif(condition2){  
    //code to be executed if condition2 is true  
} else{  
    //code to be executed if both conditions are false  
}
```

- Example:

```
<?php  
$d = date("D");  
if($d == "Fri"){  
    echo "Have a nice weekend!";  
} elseif($d == "Sun"){  
    echo "Have a nice Sunday!";  
} else{  
    echo "Have a nice day!";  
}  
?>
```

PHP Conditional Statements (Cont...)

- The switch . . .case statement

```
switch(n){  
  case label1:  
    //code to be executed if n=label1  
    break;  
  case label2:  
    //code to be executed if n=label2  
    break;  
  . . .  
  default:  
    //code to be executed if n is different from all labels  
}
```

PHP Conditional Statements (Cont...)

- The switch . . .case statement example

```
<?php
$today = date("D");
switch($today){
    case "Mon":
        echo "Today is Monday. Clean your house.";
        break;
    case "Tue":
        echo "Today is Tuesday. Buy some food.";
        break;
    case "Wed":
        echo "Today is Wednesday. Visit a doctor.";
        break;
    case "Thu":
        echo "Today is Thursday. Repair your car.";
        break;
```

```
    case "Fri":
        echo "Today is Friday. Party tonight.";
        break;
    case "Sat":
        echo "Today is Saturday. Its movie time.";
        break;
    case "Sun":
        echo "Today is Sunday. Do some rest.";
        break;
    default:
        echo "No information available for that
day.";
        break;
}
?>
```

PHP Ternary Operator (?)

- The if . . . else statement

```
<?php
$age=18;
if($age < 18){
    echo 'Child'; // Display Child if age is less than 18
} else{
    echo 'Adult'; // Display Adult if age is greater than or equal to 18
}
?>
```

- The ternary operator statement

```
echo ($age < 18) ? 'Child' : 'Adult';
```


PHP Loops

- Loops are used to execute the same block of code again and again, until a certain condition is met.
- The basic idea behind a loop is to automate the repetitive tasks within a program to save the time and effort.
- PHP supports four different types of loops:
 - while – loops through a block of code until the condition is evaluate to true.
 - do . . . while – the block of code executes once and then condition is evaluated. If the condition is true the statement is repeated as long as the specified condition is true.
 - for – loops through a block of code until the counter reaches a specified number.
 - foreach – loops through a block of code for each element in an array.

PHP Loops (Cont...)

- While Loop

- Syntax:

```
while(condition){  
    // Code to be executed  
}
```

- Example:

```
<?php  
$i = 1;  
while($i <= 3){  
    $i++;  
    echo "The number is " . $i . "<br>";  
}  
?>
```

PHP Loops (Cont...)

- do...while Loop

- Syntax:

```
do{  
    // Code to be executed  
}  
while(condition);
```

- Example:

```
<?php  
$i = 1;  
do{  
    $i++;  
    echo "The number is " . $i . "<br>";  
}  
while($i <= 3);  
?>
```

PHP Loops (Cont...)

- for Loop

- Syntax:

```
for(initialization; condition; increment){  
    // Code to be executed  
}
```

- Example:

```
<?php  
for($i=1; $i<=3; $i++){  
    echo "The number is " . $i . "<br>";  
}  
?>
```

PHP Loops (Cont...)

- foreach Loop

- Syntax:

```
foreach($array as $value){  
    // Code to be executed  
}
```

- Example:

```
<?php  
$colors = array("Red", "Green", "Blue");  
foreach($colors as $value){  
    echo $value . "<br>";  
}  
?>
```

PHP Functions

- A function is a self-contained block of code that performs a specific task.
- PHP function is a piece of code that can be reused many times. It can take input as argument list and return value.
- PHP has a huge collection of internal or built-in functions that you can call directly within your PHP scripts to perform a specific task, like `gettype()`, `print_r()`, `var_dump`, etc. ()
- In addition to the built-in functions, PHP also allows you to define your own functions.
- It is a way to create reusable code packages that perform specific tasks and can be kept and maintained separately from main program.

PHP Functions (Cont...)

- Advantage of PHP Functions
 - Functions reduces the repetition of code within a program
 - It allows you to extract commonly used block of code into a single component.
 - Now you can perform the same task by calling this function wherever you want within your script without having to copy and paste the same block of code again and again.
 - Functions makes the code much easier to maintain
 - Since a function created once can be used many times, so any changes made inside a function automatically implemented at all the places without touching the several files.
 - Functions makes it easier to eliminate the errors
 - When the program is subdivided into functions, if any error occur you know exactly what function is causing the error and where to find it.
 - Therefore, fixing errors becomes much easier.
 - Functions can be reused in other application
 - A function is separated from the rest of the script.
 - So, it's easy to reuse the same function in other applications just by including the php file containing those functions.

PHP User-defined Functions

- We can declare and call user-defined functions easily.
- Syntax

```
function functionname(){  
    //code to be executed  
}
```

 - [Note: Function name must be start with letter and underscore only like other labels in PHP. It can't be start with numbers or special symbols.]
- Example

```
<?php  
function sayHello(){  
    echo "Hello PHP Function";  
}  
sayHello();//calling function  
?>
```


PHP Functions Examples

- Simple PHP Functions Example

```
<?php
// Defining function
function whatIsToday(){
    echo "Today is " . date("l");
}
whatIsToday();
?>
```

PHP Functions Examples (Cont...)

- PHP Functions with parameters

```
<?php
// Defining function
function getSum($num1, $num2){
    $sum = $num1 + $num2;
    echo "Sum of the two numbers $num1 and $num2 is : $sum";
}
// Calling function
getSum(10, 20);
?>
```

PHP Date() Function

- Example:

```
<?php
// Prints the day
echo date("l") . "<br>";
```

```
// Prints the day, date, month, year, time, AM or PM
echo date("l jS \of F Y h:i:s A");
?>
```

Parameter Values

- d - The day of the month (from 01 to 31)
- D - A textual representation of a day (three letters)
- j - The day of the month without leading zeros (1 to 31)
- l (lowercase 'l') - A full textual representation of a day
- N - The ISO-8601 numeric representation of a day (1 for Monday, 7 for Sunday)
- S - The English ordinal suffix for the day of the month (2 characters st, nd, rd or th. Works well with j)
- w - A numeric representation of the day (0 for Sunday, 6 for Saturday)
- z - The day of the year (from 0 through 365)
- W - The ISO-8601 week number of year (weeks starting on Monday)
- F - A full textual representation of a month (January through December)
- m - A numeric representation of a month (from 01 to 12)
- M - A short textual representation of a month (three letters)
- n - A numeric representation of a month, without leading zeros (1 to 12)
- t - The number of days in the given month
- L - Whether it's a leap year (1 if it is a leap year, 0 otherwise)

PHP Date() Function (Cont...)

- o - The ISO-8601 year number
- Y - A four digit representation of a year
- y - A two digit representation of a year
- a - Lowercase am or pm
- A - Uppercase AM or PM
- B - Swatch Internet time (000 to 999)
- g - 12-hour format of an hour (1 to 12)
- G - 24-hour format of an hour (0 to 23)
- h - 12-hour format of an hour (01 to 12)
- H - 24-hour format of an hour (00 to 23)
- i - Minutes with leading zeros (00 to 59)
- s - Seconds, with leading zeros (00 to 59)
- u - Microseconds (added in PHP 5.2.2)
- e - The timezone identifier (Examples: UTC, GMT, Atlantic/Azores)
- I (capital i) - Whether the date is in daylight savings time (1 if Daylight Savings Time, 0 otherwise)
- O - Difference to Greenwich time (GMT) in hours (Example: +0100)
- P - Difference to Greenwich time (GMT) in hours:minutes (added in PHP 5.1.3)
- T - Timezone abbreviations (Examples: EST, MDT)
- Z - Timezone offset in seconds. The offset for timezones west of UTC is negative (-43200 to 50400)
- c - The ISO-8601 date (e.g. 2013-05-05T16:34:42+00:00)
- r - The RFC 2822 formatted date (e.g. Fri, 12 Apr 2013 12:01:05 +0200)
- U - The seconds since the Unix Epoch (January 1 1970 00:00:00 GMT)

PHP Arrays

- PHP array is an ordered map (contains value on the basis of key). It is used to hold multiple values of similar type in a single variable.
- Advantage of PHP Array
 - Less Code: We don't need to define multiple variables.
 - Easy to traverse: By the help of single loop, we can traverse all the elements of an array.
 - Sorting: We can sort the elements of array.
- Storing the colors one by one in a variable could look like this:

```
<?php
$color1 = "red";
$color2 = "green";
$color3 = "blue";
?>
```
- The following example shows how an indexed array is created:

```
<?php
$colors = array("red", "green", "blue");
?>
```

PHP Arrays (Cont...)

- Types of Arrays
 - **Indexed Array** – array with a numeric key
 - **Associative Array** – array where each key has its own specific value
 - **Multidimensional Array** – array containing one or more arrays within itself
- **Indexed Arrays**
 - An indexed (or numeric) array stores each array element with a numeric index.
 - Storing the colors one by one in a variable could look like this:

```
<?php
$color1 = "red";
$color2 = "green";
$color3 = "blue";
?>
```
 - The following example shows how an indexed array is created:

```
<?php
$colors = array("red", "green", "blue");
?>
```

PHP Arrays (Cont...)

- **Associative Arrays**

- In an associative array, the keys assigned to values can be arbitrary and user defined strings.
- In the following example the array uses keys instead of index numbers:

```
<?php
$ages = array("Peter"=>22, "Clark"=>32, "John"=>28);
?>
```

- The following example displays the associative array:

```
<?php
$ages = array("Peter"=>22, "Clark"=>32, "John"=>28);
foreach($ages as $key_arr=>$val_arr)
echo $key_arr . "=" . $val_arr . "<br>";
?>
```

PHP Arrays (Cont...)

- **Multidimensional Arrays**

- An array in which each element can also be an array and each element in the sub-array can be an array or further contain array within itself and so on.

```
<?php
$contacts = array(
    array(
        "name" => "Peter Parker",
        "email" => "peterparker@mail.com",
    ),
    array(
        "name" => "Harry Potter",
        "email" => "harrypotter@mail.com",
    )
);
echo "His/Her name is ".$contacts[0]["name"] . " and email-id is:" . $contacts[0]["email"];
echo "<br>";
echo "His/Her name is ".$contacts[1]["name"] . " and email-id is:" . $contacts[1]["email"];
?>
```


Sorting PHP Arrays

- PHP comes with a number of built-in functions designed specifically for sorting array elements in different ways like alphabetically or numerically in ascending or descending order.
- `sort()` and `rsort()` – for sorting indexed arrays
- `asort()` and `arsort()` – for sorting associative arrays by value
- `ksort()` and `krsort()` – for sorting associative arrays by key

Sorting PHP Arrays (Cont...)

- Sorting Indexed Arrays:

```
<?php
// Define array
$colors = array("Red", "Green", "Blue", "Yellow");
// Sorting and printing array
sort($colors); // Ascending Order
//
print_r($colors);
//
echo "<br>";
rsort($colors); // Descending Order
print_r($colors);
?>
```

- Output

```
Array ( [0] => Blue [1] => Green [2] => Red [3] => Yellow )
Array ( [0] =>Yellow [1] => Red [2] => Green [3] => Blue )
```

Sorting PHP Arrays (Cont...)

- Sorting Associative Arrays by Value:

```
<?php
// Define array
$age = array("Peter"=>20, "Harry"=>14, "John"=>45, "Clark"=>35);
// Sorting array by value and print
asort($age); // Ascending Order
print_r($age);
echo "<br>";
arsort($age); // Descending Order
print_r($age);
?>
```

- Output:

```
Array ( [Harry] => 14 [Peter] => 20 [Clark] => 35 [John] => 45 )
Array ( [John] => 45 [Clark] => 35 [Peter] => 20 [Harry] => 14 )
```

Sorting PHP Arrays (Cont...)

- Sorting Associative Arrays by Key:

```
<?php
// Define array
$age = array("Peter"=>20, "Harry"=>14, "John"=>45, "Clark"=>35);
// Sorting array by key and print
ksort($age); // Ascending Order
print_r($age);
echo "<br>";
krsort($age); // Descending Order
print_r($age);
?>
```

- Output:

```
Array ( [Clark] => 35 [Harry] => 14 [John] => 45 [Peter] => 20 )
Array ( [Peter] => 20 [John] => 45 [Harry] => 14 [Clark] => 35 )
```

Creating Class and Objects

- OOP

- In OOP, there's a concept of "class", which is used to model or map a real-world entity to a template of data (properties) and functionality (methods).
- An "object" is an instance of a class, and you can create multiple instances of the same class. For example, there is a single Person class, but many person objects can be instances of this class—dan, zainab, hector, etc.
- The class defines properties. For example, for the Person class, we might have name, age, and phoneNumber. Then each person object will have its own values for those properties.

Creating Class and Objects (Cont...)

- PHP Class

- A class is a template which represents a real-world entity, and it defines properties and methods of the entity.

```
<?php
class Employee
{
    private $first_name;
    private $last_name;
    private $age;
    public function __construct($first_name,
    $last_name, $age)
    {
        $this->first_name = $first_name;
        $this->last_name = $last_name;
        $this->age = $age;
    }
    public function getFirstName()
    {
```

```
        return $this->first_name;
    }
    public function getLastName()
    {
        return $this->last_name;
    }
    public function getAge()
    {
        return $this->age;
    }
}
?>
```

- The class Employee statement in the first line defines the Employee class. Then, we go on to declare the properties, the constructor, and the other class methods.

Creating Class and Objects (Cont...)

- Class Properties in PHP
 - We could think of class properties as variables that are used to hold information about the object.
 - In the above example, we've defined three properties—first_name, last_name, and age.
 - In most cases, class properties are accessed via instantiated objects.
 - These properties are private, which means they can only be accessed from within the class.
- Constructors for PHP Classes
 - A constructor is a special class method which is called automatically when we instantiate an object.
 - A constructor is used to initialize object properties when the object is being created.
 - We can define a constructor by defining the `__construct` method.

Creating Class and Objects (Cont...)

- Methods for PHP Classes
 - We can think of class methods as functions that perform specific actions associated with objects. In most cases, they are used to access and manipulate object properties and perform related operations.
 - In the above example, we've defined the `getLastName` method, which returns the last name associated with the object.

Creating Class and Objects (Cont...)

- Object in PHP

- when we want to use a class, we need to instantiate it, and the end result is an object.
- So we could think of a class as a blueprint, and an object is an actual thing that you can work with.
- In the context of the Employee class which we've just created in the previous section, let's see how to instantiate an object of that class.

```
<?php
$objEmployee = new Employee('Bob', 'Smith', 30);
echo $objEmployee->getFirstName(); // print 'Bob'
echo $objEmployee->getLastName(); // prints 'Smith'
echo $objEmployee->getAge(); // prints '30'
?>
```

- need to use the new keyword when we want to instantiate an object of any class along with its class name, and we'll get back a new object instance of that class.

Creating Class and Objects (Cont...)

- Object in PHP

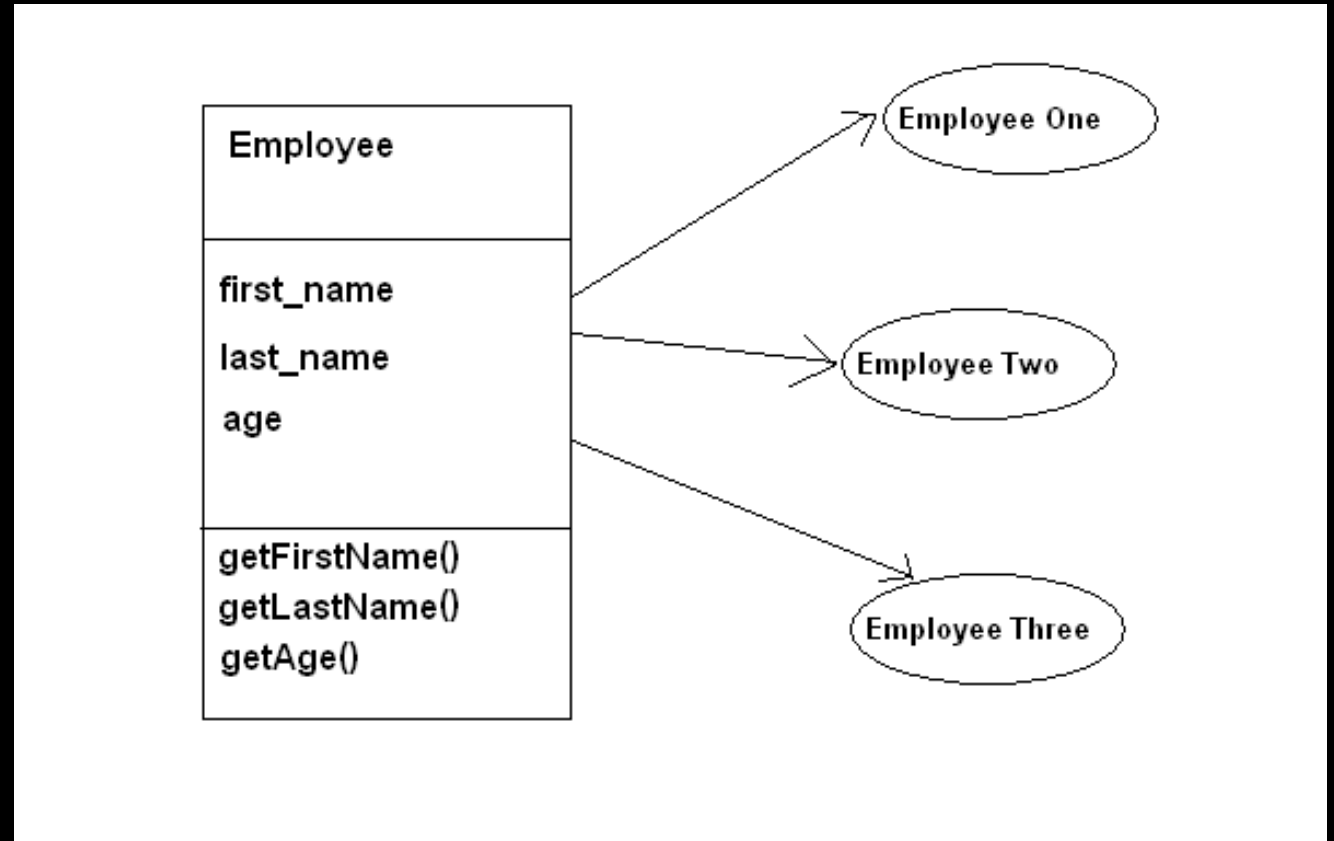
- If a class has defined the `__construct` method and it requires arguments, we need to pass those arguments when we instantiate an object.
- In our case, the `Employee` class constructor requires three arguments, and thus we've passed these when we created the `$objEmployee` object.
- The `__construct` method is called automatically when the object is instantiated.
- We've called class methods on the `$objEmployee` object to print the information which was initialized during object creation.
- We can create multiple objects of the same class, as shown in the following snippet.

```
<?php
$objEmployeeOne = new Employee('Bob', 'Smith', 30);
    echo $objEmployeeOne->getFirstName(); // prints 'Bob'
    echo $objEmployeeOne->getLastName(); // prints 'Smith'
    echo $objEmployeeOne->getAge(); // prints '30'
$objEmployeeTwo = new Employee('John', 'Smith', 34);
    echo $objEmployeeTwo->getFirstName(); // prints 'John'
    echo $objEmployeeTwo->getLastName(); // prints 'Smith'
    echo $objEmployeeTwo->getAge(); // prints '34'

?>
```

Creating Class and Objects (Cont...)

- Object in PHP
 - The following image is a graphical representation of the Employee class and some of its instances.
 - A class is a blueprint which we can use to create structured objects.



PHP Form

- When we develop a website or a web application, we often have to create forms to take input from users, like a Login form or a Registration form.
- Creating a form on the webpage is accomplished using HTML, while PHP serves as a transport for those values from the webpage to the server and then in further processing those values.
- PHP provides two superglobals `$_GET` and `$_POST` for collecting form-data for processing.

- Example:

```
<html>
  <body>
    <form action="form-handler.php" method="POST">
      Name: <input type="text" name="name"> <br/>
      Email: <input type="text" name="email"> <br/>
      <input type="submit">
    </form>
  </body>
</html>
```

- In the code above, we have used the `<form>` tag to create an HTML form, with input fields for Name and Email along with submit button to submit the form-data.

Accessing Form Elements

- In the <form> tag, two attributes, action and method are used.
 1. **action:** Using this attribute, we can specify the name of the file which will collect and handle the form-data. In the example above, we have provided name of a Php file.
 2. **method:** This attribute specify the means of sending the form-data, whether it will be submitted via **POST** method or **GET** method.
- When a user submits a form, the values from the input fields are stored in an array, like array(key1=>value1, key2=>value2,...) and then passed on to the destination(Php file) specified in the action attribute of the <form> tag.
- Using **POST** method:
 - If we specify the form method to be POST, then the form-data is sent to the server using the HTTP POST method.
 - When we use the POST method, the array of key-value pair(the form-data), coming from the HTML form are sent as part of the HTTP request, hence they are invisible to the user.
 - Also, there is no character limit for the information/data being transmitted.
 - POST method also supports multipart form-data upload which is used for file upload.
 - Use the POST method while working on any PHP web application/project.

Accessing Form Elements (cont...)

- Using POST method, Example:

```
<html>
  <body>
    <form action="form-handler.php" method="POST">
      Name: <input type="text" name="name"> <br/>
      Age: <input type="text" name="age"> <br/>
      <input type="submit">
    </form>
  </body>
</html>
```

- The Php file form-handler.php will look like,

```
<?php
// name attribute of the input field goes inside the
// square brackets of $_POST superglobal
$name = $_POST["name"];
$age = $_POST["age"];
echo "Your name is ". $name . " and you are ". $age . " years old";
?>
```

Accessing Form Elements (cont...)

- POST Method Advantages & Disadvantages
 - It is more secure than GET because user-entered information is never visible in the URL query string or in the server logs.
 - There is a much larger limit on the amount of data that can be passed and one can send text data as well as binary data (uploading a file) using POST.
 - Since the data sent by the POST method is not visible in the URL, it is not possible to bookmark the page with specific query.
 - PHP provides the super global variable `$_POST` to access all the information sent via post method or submitted through an HTML form using the method="POST"

Accessing Form Elements (cont...)

- Using **GET** method:
 - In case of GET method, form-data is submitted as URL parameters, i.e. all the values entered in the form fields by the user are sent to the action script, appended in the URL.
 - We have two input fields in above form, one is name and the other one is age. When we click on submit, we will be redirected to the following URL, form-handler.php?name=Studytonight&age=5, with the form-data appended to the URL.
 - Sending the form-data as URL parameters proves out useful at times as you can easily bookmark links with form-data, but for appending parameters in a URL there is a limit of 2000 characters, hence for forms with large number of fields, it is not suggested, as some data might get lost or the form submission may lead to error.

Accessing Form Elements (cont...)

- Using GET method Example:

```
<html>
  <body>
    <form action="form-handler.php" method="GET">
      Name: <input type="text" name="name"> <br/>
      Age: <input type="text" name="age"> <br/>
      <input type="submit">
    </form>
  </body>
</html>
```

- The Php file form-handler.php will look like,

```
<?php
// name attribute of the input field goes inside the
// square brackets of $_GET superglobal
$name = $_GET["name"];
$age = $_GET["age"];
echo "Your name is ". $name . " and you are ". $age . " years old";
?>
```

Accessing Form Elements (cont...)

- GET Method Advantages & Disadvantages
 - Since the data sent by the GET method are displayed in the URL, it is possible to bookmark the page with specific query string values.
 - The GET method is not suitable for passing sensitive information such as the username and password, because these are fully visible in the URL query string as well as potentially stored in the client browser's memory as a visited page.
 - Because the GET method assigns data to a server environment variable, the length of the URL is limited. So there is a limitation for the total data to be sent.
 - PHP provides the super global variable `$_GET` to access all the information sent either through the URL or submitted through an HTML form using the `method="GET"`

PHP Form Validation

- An HTML form contains various input fields such as text box, checkbox, radio buttons, submit button, and checklist, etc. These input fields need to be validated, which ensures that the user has entered information in all the required fields and also validates that the information provided by the user is valid and correct.
- PHP validates the data at the server-side, which is submitted by HTML form. You need to validate a few things:
 1. Empty String
 2. Validate String
 3. Validate Numbers
 4. Validate Email
 5. Validate URL
 6. Input length

PHP Form Validation cont...

- Empty String

- The code below checks that the field is not empty. If the user leaves the required field empty, it will show an error message. Put these lines of code to validate the required field.

```
if (empty ($_POST["name"])) {  
    $errMsg = "Error! You didn't enter the Name.";  
    echo $errMsg;  
} else {  
    $name = $_POST["name"];  
}
```

- Validate String

- The code below checks that the field will contain only alphabets and whitespace, for example - name. If the name field does not receive valid input from the user, then it will show an error message:

```
$name = $_POST ["Name"];  
if (!preg_match ("/^[a-zA-z]*$/", $name) ) {  
    $ErrMsg = "Only alphabets and whitespace are allowed.";  
    echo $ErrMsg;  
} else {  
    echo $name;  
}
```

PHP Form Validation cont...

- Validate Number

- The below code validates that the field will only contain a numeric value. For example - Mobile no. If the Mobile no field does not receive numeric data from the user, the code will display an error message:

```
$mobilenno = $_POST ["Mobile_no"];  
if (!preg_match ("/^[0-9]*$/", $mobilenno) ){  
    $ErrMsg = "Only numeric value is allowed.";  
    echo $ErrMsg;  
} else {  
    echo $mobilenno;  
}
```

- Validate Email

- A valid email must contain @ and . symbols. PHP provides various methods to validate the email address. Here, we will use regular expressions to validate the email address.

```
$email = $_POST ["Email"];  
$pattern = "^[_a-z0-9-]+(\\.[_a-z0-9-]+)*@[a-z0-9-]+(\\.[a-z0-9-]+)*(\\.[a-z]{2,3})$^";  
if (!preg_match ($pattern, $email) ){  
    $ErrMsg = "Email is not valid.";  
    echo $ErrMsg;  
} else {  
    echo "Your valid email address is: " . $email;  
}
```

PHP Form Validation cont...

- Input Length Validation

- The input length validation restricts the user to provide the value between the specified range, for Example - Mobile Number. A valid mobile number must have 10 digits.

```
$mobilenumber = $_POST ["Mobile"];
$length = strlen ($mobilenumber);
if ( $length < 10 && $length > 10) {
    $ErrMsg = "Mobile must have 10 digits.";
    echo $ErrMsg;
} else {
    echo "Your Mobile number is: " . $mobilenumber;
}
```

- Validate URL

- The below code validates the URL of website provided by the user via HTML form. If the field does not contain a valid URL, the code will display an error message, i.e., "URL is not valid".

```
$websiteURL = $_POST["website"];
if (!preg_match("/\b(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?~_!|!.,;]*[-a-z0-9+&@#\/%?~_~_!|!.,;]/i",$website)) {
    $websiteErr = "URL is not valid";
    echo $websiteErr;
} else {
    echo "Website URL is: " . $websiteURL;
}
```

PHP Form Validation cont...

- Button Click Validate

- The below code validates that the user click on submit button and send the form data to the server one of the following method - get or post.

```
if (isset ($_POST['submit'])) {  
    echo "Submit button is clicked.";  
    if ($_SERVER["REQUEST_METHOD"] == "POST") {  
        echo "Data is sent using POST method ";  
    }  
} else {  
    echo "Data is not submitted";  
}
```

Cookies and Sessions

- Cookies
 - Cookies are data, stored in small text files, on your computer.
 - When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user. Cookies were invented to solve the problem "how to remember information about the user".
 - When a user visits a web page, his/her name can be stored in a cookie. Next time the user visits the page, the cookie "remembers" his/her name. Cookies are saved in name-value pairs like:
 - username = John Doe
 - When a browser requests a web page from a server, cookies belonging to the page is added to the request. This way the server gets the necessary data to "remember" information about users.
 - Cookie can be created, sent and received at server end.
 - Note: PHP Cookie must be used before `<html>` tag.

Cookies cont...



- PHP setcookie() function:
 - PHP setcookie() function is used to set cookie with HTTP response. Once cookie is set, we can access it by `$_COOKIE` superglobal variable.
 - Syntax:

```
bool setcookie ( string $name [, string $value [, int $expire = 0 [, string $path  
[, string $domain [, bool $secure = false [, bool $httponly = false ]]]]] )
```
 - Example:

```
setcookie("CookieName", "CookieValue");/* defining name and value only*/  
setcookie("CookieName", "CookieValue", time()+1*60*60);//using expiry in 1 hour(1*60*60  
seconds or 3600 seconds)  
setcookie("CookieName", "CookieValue", time()+1*60*60, "/mypath/", "mydomain.com", 1);
```

Cookies cont...

- PHP \$_COOKIE
 - PHP \$_COOKIE superglobal variable is used to get cookie.
 - Example:
\$value=\$_COOKIE["CookieName");//returns cookie value
 - PHP Cookie Example:

```
<?php
setcookie("user", "Sonoo");
?>
<html>
<body>
<?php
if(!isset($_COOKIE["user"])) {
    echo "Sorry, cookie is not found!";
} else {
    echo "<br/>Cookie Value: " . $_COOKIE["user"];
}
?>
</body>
</html>
```

Cookies cont...

- PHP Delete Cookie

- If you set the expiration date in past, cookie will be deleted.

- Example:

```
<?php  
setcookie ("CookieName", "", time() - 3600); // set the expiration date to one hour ago  
?>
```

Session and State

- Web is Stateless, that means the web server does not know who you are and what you do, because the HTTP address doesn't maintain state.
- If user inserts some information, and move to the next page, that data will be lost, and user would not be able to retrieve the information. so, we need to store the information about user.
- Session is a State Management Technique. A Session can store the value on the Server. It can support any type of object to be stored along with our own custom objects.
- A session is one of the best techniques for State Management because it stores the data as client-based, in other words the data is stored for every user separately and the data is secured also because it is on the server.
- By default, session variables last until the user closes the browser.

Creating a Session

- Simply by calling the `session_start()` function you can create a new session. Session variables are set with the PHP global variable: `$_SESSION`.
- Syntax :
`session_start();`
- This is the initialization of session. Each and every page at the top of script, we must initiate the session, if we need any session value.
- Syntax:
`$_SESSION['session_name']="value";`
- Example:

```
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// Set session variables
$_SESSION["name"] = "Ram";
$_SESSION["address"] = "Kathmandu";
?>
</body>
</html>
```

Retrieving the value from session

- we can retrieve the value in any page if we store the value in session, session carries the values from one page to another

- Syntax :

`$_SESSION['session_name']`

- Example:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
```

```
<?php
// Echo session variables that were set
on previous page
echo "Hello " . $_SESSION["name"] .
".<br>";
echo "Your address is " .
$_SESSION["address"] ;
?>
</body>
</html>
```

Removing session and session variables

- we can also destroy our session after we finished its requirement.
- To destroy a specific session variable simply unset the corresponding key of the `$_SESSION` array. Use `session_destroy()` function to destroy all the session variables.
- If you want to destroy a single session variable then use `unset()` function.
- Syntax:
 - `session_destroy();` //destroy all the session
 - `unset($_SESSION['session_name']);`
//unset single variable

- Example:

```
<?php
session_start(); ?>
<!DOCTYPE html>
<html>
<body>
<?php
// remove name variables
unset($_SESSION['name']);
// destroy the session
session_destroy();
?>
</body>
</html>
```

Advantages and disadvantages of session

- Advantages

- It helps to maintain user states and data to all over the application.
- It can easily be implemented, and we can store any kind of object.
- Stores every client data separately.
- Session is secure and transparent from user.

- Disadvantages

- Performance overhead in case of large volume of user, because of session data stored in server memory.
- Overhead involved in serializing and De-Serializing session Data. Because In case of StateServer and SQLServer session mode we need to serialize the object before store.
- The problem with sessions is that when you close your browser you also lose the session so, if you had a site requiring a login, this couldn't be saved as a session like it could as a cookie, and the user would be forced to re-login every time they visit.

Cookies Vs. Session

Cookie	Session
Cookie ends depending on the lifetime you set for it	A session ends when a user closes his browser
The official maximum cookie size is 4KB	Within-session you can store as much data as you like. The only limits you can reach is the maximum memory a script can consume at one time, which is 128MB by default
There is no function named unsetcookie()	Session_destroy(); is used to destroy all registered data or to unset some

Working with PHP and MySQL

- PHP will work with virtually all database software, including Oracle and Sybase but most commonly used is freely available MySQL database.
- MySQL
 - MySQL is an open-source relational database management system (RDBMS).
 - It is the most popular database system used with PHP. MySQL is developed, distributed, and supported by Oracle Corporation.
 - Features:
 - The data in a MySQL database are stored in tables which consists of columns and rows.
 - MySQL is a database system that runs on a server.
 - MySQL is ideal for both small and large applications.
 - MySQL is very fast, reliable, and easy to use database system. It uses standard SQL
 - MySQL compiles on a number of platforms.
 - There are three ways of working with MySQL and PHP
 - MySQLi (object-oriented)
 - MySQLi (procedural)
 - PDO

Connecting to MySQL database using PHP

- Using MySQLi object-oriented procedure:
 - We can use the MySQLi object-oriented procedure to establish a connection to MySQL database from a PHP script.
 - Syntax:

```
<?php
    $servername = "localhost";
    $username = "username";
    $password = "password";
    // Creating connection
    $conn = new mysqli($servername, $username, $password);
    // Checking connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }
    echo "Connected successfully";
?>
```

Explanation: We can create an instance of the mysqli class providing all the necessary details required to establish the connection such as host, username, password etc. If the instance is created successfully then the connection is successful otherwise there is some error in establishing connection.

Connecting to MySQL database using PHP (cont...)

- Using MySQLi procedural procedure:

- Syntax:

```
<?php
    $servername = "localhost";
    $username = "username";
    $password = "password";
    // Creating connection
    $conn = mysqli_connect($servername, $username, $password);
    // Checking connection
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }
    echo "Connected successfully";
?>
```

Explanation: In MySQLi procedural approach instead of creating an instance we can use the `mysqli_connect()` function available in PHP to establish a connection. This function takes the information as arguments such as host, username, password, database name etc. This function returns MySQL link identifier on successful connection or FALSE when failed to establish a connection.

Connecting to MySQL database using PHP (cont...)

- Using PDO procedure:

- PDO stands for PHP Data Objects. That is, in this method we connect to the database using data objects in PHP as described below:

- Syntax:

```
<?php
    $servername = "localhost";
    $username = "username";
    $password = "password";
    try {
        $conn = new PDO("mysql:host=$servername;dbname=myDB", $username, $password);
        // setting the PDO error mode to exception
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        echo "Connected successfully";
    }
    catch(PDOException $e)
    {
        echo "Connection failed: " . $e->getMessage();
    }
?>
```

Explanation: The exception class in PDO is used to handle any problems that may occur in our database queries. If an exception is thrown within the try{ } block, the script stops executing and flows directly to the first catch(){ } block.

Closing A Connection

- When we establish a connection to MySQL database from a PHP script , we should also disconnect or close the connection when our work is finished.
- Using MySQLi object-oriented procedure
 - Syntax
`$conn->close();`
- Using MySQLi procedural procedure
 - Syntax
`mysqli_close($conn);`
- Using PDO procedure
 - Syntax
`$conn = null;`

MySQL CREATE TABLE

- A table is used to organize data in the form of rows and columns and used for both storing and displaying records in the structure format.
- It is similar to worksheets in the spreadsheet application.
- A table creation command requires three things:
 - Name of the table
 - Names of fields
 - Definitions for each field
- MySQL allows us to create a table into the database mainly in two ways:
 - MySQL Command Line
 - MySQL Workbench (Using workbench such as phpmyadmin)

MySQL CREATE TABLE (Cont...)

- MySQL Command Line

- MySQL allows us to create a table into the database by using the CREATE TABLE command.

- Syntax

```
CREATE TABLE table_name (column_name column_type);
```

- Example

```
CREATE TABLE tutorials_tbl(  
    tutorial_id INT NOT NULL AUTO_INCREMENT,  
    tutorial_title VARCHAR(100) NOT NULL,  
    tutorial_author VARCHAR(40) NOT NULL,  
    submission_date DATE,  
    PRIMARY KEY ( tutorial_id )  
);
```


MySQL CREATE TABLE (Cont...)

- Creating Tables Using PHP Script

- PHP uses `mysqli_query()` or `mysql_query()` function to create a MySQL table.
- This function takes two parameters and returns `TRUE` on success or `FALSE` on failure.

- Syntax

`$mysqli->query($sql,$resultmode)`

Here,

- `$sql`: Required - SQL query to create a MySQL table.
- `$resultmode`: Optional - Either the constant `MYSQLI_USE_RESULT` or `MYSQLI_STORE_RESULT` depending on the desired behavior. By default, `MYSQLI_STORE_RESULT` is used.

MySQL CREATE TABLE (Cont...)

- Creating Tables Using PHP Script Example

```
<html>
<head>
  <title>Creating MySQL Table</title>
</head>
<body>
  <?php
    $dbhost = 'localhost';
    $dbuser = 'root';
    $dbpass = 'root@123';
    $dbname = 'TUTORIALS';
    $mysqli = new mysqli($dbhost, $dbuser,
                        $dbpass, $dbname);
    if($mysqli->connect_errno ) {
      printf("Connect failed: %s<br />", $mysqli-
        >connect_error);
      exit();
    }
    printf('Connected successfully.<br />');
    $sql = "CREATE TABLE tutorials_tbl ( "
      "tutorial_id INT NOT NULL AUTO_INCREMENT,
```

```
".
    "tutorial_title VARCHAR(100) NOT NULL, ".
    "tutorial_author VARCHAR(40) NOT NULL, ".
    "submission_date DATE, ".
    "PRIMARY KEY ( tutorial_id )); ";
    if ($mysqli->query($sql)) {
      printf("Table tutorials_tbl created
        successfully.<br />");
    }
    if ($mysqli->errno) {
      printf("Could not create table: %s<br />",
        $mysqli->error);
    }
    $mysqli->close();
  ?>
</body>
</html>
```

MySQL Drop TABLE

- Drop Tables Using PHP Script

- Syntax:

- DROP TABLE table_name ;

- Example

- ```
<html>
 <head><title>Dropping MySQL
Table</title></head>
 <body>
 <?php
 $dbhost = 'localhost';
 $dbuser = 'root';
 $dbpass = '';
 $dbname = 'TUTORIALS';
 $mysqli = new mysqli($dbhost, $dbuser,
 $dbpass, $dbname);
 if($mysqli->connect_errno) {
 printf("Connect failed: %s
"
$mysqli->connect_error);
```

```
 exit();
 }
 printf('Connected successfully.
');
 if ($mysqli->query("Drop Table tutorials_tbl"))
 {
 printf("Table tutorials_tbl dropped
 successfully.
");
 }
 if ($mysqli->errno) {
 printf("Could not drop table: %s
",
 $mysqli->error);
 }
 $mysqli->close();
 ?>
 </body>
</html>
```

# MySQL Insert query

- Insert data into a MySQL table:
  - We would need to use the SQL INSERT INTO command. We can insert data into the MySQL table by using the mysql> prompt or by using any script like PHP.
  - Syntax

```
INSERT INTO table_name (field1, field2,...fieldN)
VALUES (value1, value2,...valueN);
```

# MySQL Select Query

- Select data from a MySQL table:

- The SQL SELECT command is used to fetch data from the MySQL database.
- Syntax

```
SELECT field1, field2,...fieldN
FROM table_name1, table_name2...
[WHERE Clause]
[OFFSET M][LIMIT N]
```

- We can use one or more tables separated by comma to include various conditions using a WHERE clause, but the WHERE clause is an optional part of the SELECT command.
- We can fetch one or more fields in a single SELECT command.
- We can specify star (\*) in place of fields. In this case, SELECT will return all the fields.
- We can specify any condition using the WHERE clause.
- We can specify an offset using OFFSET from where SELECT will start returning records. By default, the offset starts at zero.
- We can limit the number of returns using the LIMIT attribute.

# MySQL Update Query

- Update MySQL table:
  - There may be a requirement where the existing data in a MySQL table needs to be modified. We can do so by using the SQL UPDATE command. This will modify any field value of any MySQL table.
  - Syntax  
UPDATE table\_name SET field1 = new-value1, field2 = new-value2  
[WHERE Clause]
    - We can update one or more field altogether.
    - We can specify any condition using the WHERE clause.
    - We can update the values in a single table at a time.

# MySQL DELETE Query

- Delete record from MySQL table:
  - If we want to delete a record from any MySQL table, then we can use the SQL command DELETE FROM.
  - Syntax  
DELETE FROM table\_name [WHERE Clause]
    - If the WHERE clause is not specified, then all the records will be deleted from the given MySQL table.
    - We can specify any condition using the WHERE clause.
    - We can delete records in a single table at a time.
  - The WHERE clause is very useful when we want to delete selected rows in a table.